



NoSQL Databases

Subject Code: CIAEC49

B.E Sem: IV

Credits : 1



Unit I

Introduction: Why NoSQL? The Value of Relational Databases, RDBMS vs NoSQL, CAP Theorem, Impedance Mismatch, Application and Integration Databases, Attack of the Clusters, The Emergence of NoSQL.



Why NoSQL?



The Value of Relational Databases

Relational databases didn't become dominant by accident. They provide a specific set of core benefits that addressed the primary needs of enterprise computing:

- **Persistent Data Storage:** Moving data from volatile memory to stable storage.
- **Concurrency Management:** Handling multiple users at the same time.
- **Integration:** Allowing different applications to work together.
- **Standardization:** A unified model that works across different vendors.



1. Persistent Data & Storage Architecture

Memory Hierarchy:

- **Main Memory:** Fast but volatile (data is lost if power fails).
- **Backing Store:** Larger, slower, but persistent (traditionally disks, now often persistent memory).

Files vs. Databases:

- **File Systems:** Useful for simple applications (like word processors), but limited for complex data.
- **Databases:** Provide a specialized backing store that allows applications to retrieve **small segments of data** quickly from massive datasets.

The Enterprise Need: Applications require the ability to search, filter, and modify specific records without loading the entire "file" into memory.



2. Managing Concurrency

The Problem: Enterprise apps often involve many users interacting with the same data simultaneously. Without control, this leads to errors like **double-booking** or data corruption.

The Solution: Transactions: Relational databases control access through a transactional mechanism.

- **Coordination:** Ensures that if two people try to edit the same record, the system handles the conflict predictably.
- **Error Handling:** Transactions allow for **Rollbacks**. If an error occurs mid-process, the database can revert to its previous "clean" state, preventing partial or "ghost" updates.

Complexity Management: While not a perfect fix for every logic error, transactions significantly reduce the complexity developers face when writing multi-user software.

3. Integration & Shared Data

Application Silos: In large organizations, multiple teams build different applications that must work together.

Shared Database Integration:

- Applications store their data in a **single, central relational database**.
- Updates made by one application are immediately visible to all others.

Organizational Value: This approach bypasses the need for complex custom communication protocols between teams. The database acts as the "single source of truth" and handles the heavy lifting of concurrency across different systems.



4. A Unified Standard Model

Predictability: The relational model provides a consistent framework. Whether using Oracle, MySQL, or SQL Server, the core mechanisms remain familiar.

SQL (Structured Query Language): * While minor "dialects" exist, SQL is a mostly standard language.

- This allows developers and DBAs to transfer their skills across different projects and vendors easily.

Reliability: Because the model is standardized and decades old, it is perceived as a "safe" and mature choice for mission-critical enterprise data.



The Impedance Mismatch

Definition: The "Impedance Mismatch" refers to the fundamental difference between how data is structured in **application memory** versus how it is stored in a **relational database**.

The In-Memory Model: Modern programming languages use rich, nested structures, objects, lists, and associative arrays.

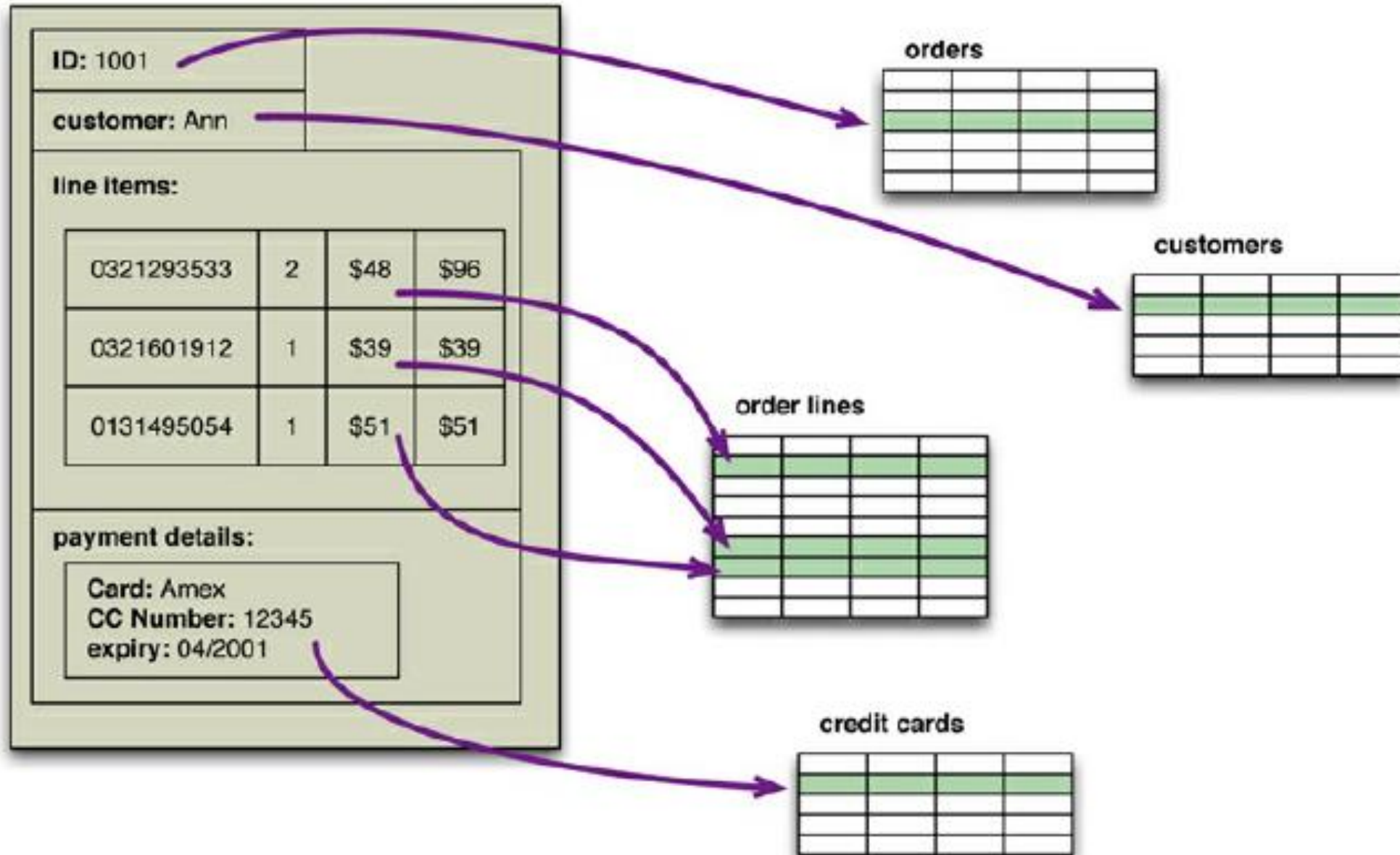
The Relational Model: Data is strictly organized into **relations (tables)** and **tuples (rows)**.

- In a relational tuple, values must be **simple** (primitive types).
- They cannot natively contain nested records or lists.

The Translation Tax: Because these two worlds speak different "languages," developers must constantly write code to translate data back and forth, leading to complexity and overhead.



The Impedance Mismatch





The Impedance Mismatch

The "Order" Example

The User's Perspective: An "Order" is a single, cohesive unit containing a customer, multiple line items, a shipping address, and payment info.

The Relational Reality: To store that single "Order," a relational database must shred it into multiple rows across several tables:

- Orders table (ID, Date, Total)
- Order_Items table (Product, Quantity, Price)
- Customers table (Name, Email)
- Addresses table (Street, City, Zip)

The Complexity: Retrieving that single Order requires complex **JOIN** operations to stitch the data back together into an in-memory object.



The Failed Revolution

The Promise: If the mismatch is the problem, why not use a database that stores objects directly to disk?

The Trend: The 1990s saw a massive shift toward Object-Oriented Programming (OOP). Many predicted that **Object-Oriented Databases (OODBMS)** would replace the relational model.

Why Relational Databases Won:

- **Integration Strength:** RDBMS proved superior at sharing data across *different* applications.
- **Standardization:** SQL provided a common language that OODBs lacked.
- **Professional Roles:** The divide between Application Developers and Database Administrators (DBAs) solidified the RDBMS as the "source of truth."

The Result: OOP succeeded in code, but OODBs faded into obscurity

Object-Relational Mapping (ORM)

The Middle Ground: Since databases wouldn't change, developers created frameworks to automate the translation.

Key Frameworks: Tools like **Hibernate** (Java) and **iBATIS/MyBatis** emerged to implement standard mapping patterns.

The Benefits:

- Removes "grunt work" (writing repetitive SQL).
- Allows developers to stay within their preferred object-oriented paradigm.

The Hidden Costs:

- **Leaky Abstractions:** If developers ignore the underlying database, they often write inefficient queries.
- **Performance Issues:** Automated mappings can result in "N+1" query problems or massive, unnecessary data loads.



Cracks in the Dominance

A Decade of Rule: Relational databases remained the undisputed kings of the 2000s enterprise world.

Emerging Pressures: Despite the convenience of ORMs, the relational model began to face new challenges that ORMs couldn't solve:

- **Scaling:** The difficulty of horizontal scaling across clusters.
- **Schema Rigidity:** The friction of changing table structures in fast-moving projects.
- **Big Data:** The sheer volume and variety of data generated by the modern web.

Conclusion: The "Impedance Mismatch" remains a core reason why developers eventually began looking for alternatives like NoSQL.



Application vs. Integration Databases

Integration Database: A single, central database used by multiple applications to share data.

- **Goal:** Ensure data consistency across different teams and systems.
- **Mechanism:** SQL acts as the universal "contract" between different applications.

Application Database: A database that is private to a single application and its development team.

- **Goal:** Decouple the data storage from the rest of the enterprise.
- **Mechanism:** The application provides an API (like Web Services) for others to access its data.

The Shift: During the 2000s, the industry began moving away from the "Integration Database" model toward "Application Databases."



Challenges of Integration Databases

- **Compromised Design:** A schema designed to satisfy 10 different applications is often too complex for any *single* application to use efficiently.
- **Rigidity and Coordination:** If one team wants to change a table structure, they must coordinate with every other team using that database, slowing down development.
- **Performance Conflicts:** An index added to help "App A" with reporting might slow down "App B" during high-speed data entry.
- **Integrity Burden:** Because the database cannot "trust" multiple external applications, all validation logic (foreign keys, checks) must be strictly enforced within the database itself.

The "Attack of the Clusters"

The Post-Dot-Com Explosion: Despite the 1990s market crash, the 2000s saw the birth of massive web properties (Google, Amazon, Facebook) operating at an unprecedented scale.

Data Growth Dimensions: Activity Tracking: Every click, link, and log was now being stored.

- **Complex Data:** Social networks, mapping data, and metadata grew into "Big Data" estates.
- **User Volume:** Systems had to serve millions of simultaneous visitors globally.

The Breaking Point: Traditional infrastructure could no longer keep up with the sheer volume of data and traffic without a fundamental change in strategy.



Scaling Up vs. Scaling Out

Scaling Up (Vertical Scaling): * **Process:** Buying bigger machines with more CPU, RAM, and Disk.

- **The Wall:** Costs grow exponentially as machines get larger; eventually, you hit physical hardware limits.

Scaling Out (Horizontal Scaling): * **Process:** Using a cluster of many small, "commodity" machines.

- **Cost Efficiency:** Cheaper to buy 100 small servers than one massive "super-server."
- **Resiliency:** If one small machine fails, the cluster stays online. In a "Scale Up" model, if the big machine fails, the whole system goes down.

Why Relational Databases Struggle with Clusters

Designed for Single Servers: RDBMS were built on the assumption that data lives on a single, reliable disk subsystem.

The Shared Disk Problem: Solutions like Oracle RAC or MS SQL Server use a "shared disk." While the *computing* is clustered, the *disk* remains a single point of failure and a performance bottleneck.

The "Sharding" Nightmare: To scale RDBMS, developers often manually split data across multiple servers (Sharding).

- **Loss of Features:** You lose cross-shard queries, referential integrity, and ACID transactions.
- **"Unnatural Acts":** The application code becomes incredibly complex because it must manually track where every piece of data lives.

The Licensing Barrier: Pricing models were designed for single-server setups, making cluster deployments prohibitively expensive and legally frustrating.



The Pioneers – Google and Amazon

The Motive: Google and Amazon reached the limits of relational technology earlier than anyone else. They needed to store massive data without the cost or bottlenecks of RDBMS.

The Opportunity: As wealthy tech giants, they had the engineering talent to "reinvent the wheel."

The Manifestos: In the mid-2000s, both companies published highly influential white papers that provided the blueprint for the NoSQL movement:

Google's BigTable (2006): A distributed storage system for managing structured data.

Amazon's Dynamo (2007): A highly available key-value storage system.

The Impact: These papers proved that it was possible to build high-performance, planet-scale databases that thrived on clusters.

The Origin of the Term "NoSQL"

The 1998 Irony: The term was first used by Carlo Strozzi for an open-source **relational** database that didn't use SQL (it used UNIX shell pipelines). It had no influence on the modern movement.

The 2009 Meetup: The modern usage was coined for a June 2009 meetup in San Francisco organized by Johan Oskarsson.

Choosing the Name: Eric Evans (Rackspace) suggested "NoSQL" because it was:

Short & Memorable: Perfect for a Twitter/X hashtag.

Unique: Very few Google search hits at the time.

The Initial "Septet": The first meetup featured early pioneers: Voldemort, Cassandra, Dynamite, HBase, Hypertable, CouchDB, and MongoDB.

Defining the "NoSQL"

Because there is no official governing body, "NoSQL" is defined by a set of common traits rather than a strict checklist:

- **Non-Relational:** They do not use the traditional table/row/tuple relational model.
- **Mostly Open-Source:** While closed-source versions exist, the movement is rooted in open-source development.
- **Cluster-Oriented:** Most are designed to scale out across many commodity servers.
- **Schema-less (or Schema-flexible):** You can add fields to records freely without a predefined structure.)
- **Not Only SQL:** A popular backronym suggesting that NoSQL is an *addition* to the database toolkit, not necessarily replacement for SQL.



- **The Conflict with ACID:** Relational databases rely on ACID (Atomicity, Consistency, Isolation, Durability) transactions. This is difficult to maintain across a large cluster.
- **Consistency Options:** NoSQL databases offer a range of consistency models (like "Eventual Consistency") to prioritize performance and availability in distributed systems.
- **The Exception – Graph Databases:** * Unlike most NoSQL stores, Graph databases aren't always focused on clusters.
 - Their primary value is a **Data Model** that handles complex, interconnected relationships better than rows and tables.

Relational is Still Relevant: Relational databases aren't going away; they remain excellent for their stability, familiarity, and feature sets.

The Paradigm Shift: Instead of asking "*Which relational database should I use?*", architects now ask "*What is the nature of my data, and which tool fits best?*"

Definition of Polyglot Persistence: Using different data storage technologies for different needs within the same organization or even the same application.

Prerequisite: This model works best when using **Application Databases** (encapsulated by services) rather than a single massive **Integration Database**.

When considering a move to NoSQL, teams are usually motivated by one (or both) of these factors:

The Scalability Need (Size & Performance):

- Handling data volumes that exceed the capacity of a single server.
- Requirement for high-velocity reads/writes across a cluster.

The Productivity Need (Development Style):

- Solving the **Impedance Mismatch** by using data structures that match the code (e.g., Documents for JSON).
- Increasing development speed through schema flexibility and more "convenient" data interaction.



Feature	Relational (RDBMS)	NoSQL
Data Model	Tabular (Rows and Columns) with strict relations.	Non-tabular (Document, Key-Value, Graph, Wide-column).
Schema	Static/Predefined: You must define the structure before adding data.	Dynamic/Schema-less: You can add data without a predefined structure.
Scaling	Vertical (Scale Up): Requires a bigger, more powerful server.	Horizontal (Scale Out): Runs on clusters of commodity hardware.
Transactions	ACID compliant: Focuses on strong consistency and integrity.	BASE compliant: Prioritizes availability and "Eventual Consistency."
Primary Goal	General-purpose storage and cross-application integration.	High-volume data, performance, and developer productivity.
Complex Queries	Highly efficient via SQL and complex JOINS.	Varies by type; often requires MapReduce or specific APIs.
Integration	Often used as an Integration Database shared by many apps.	Usually used as an Application Database private to one service.